

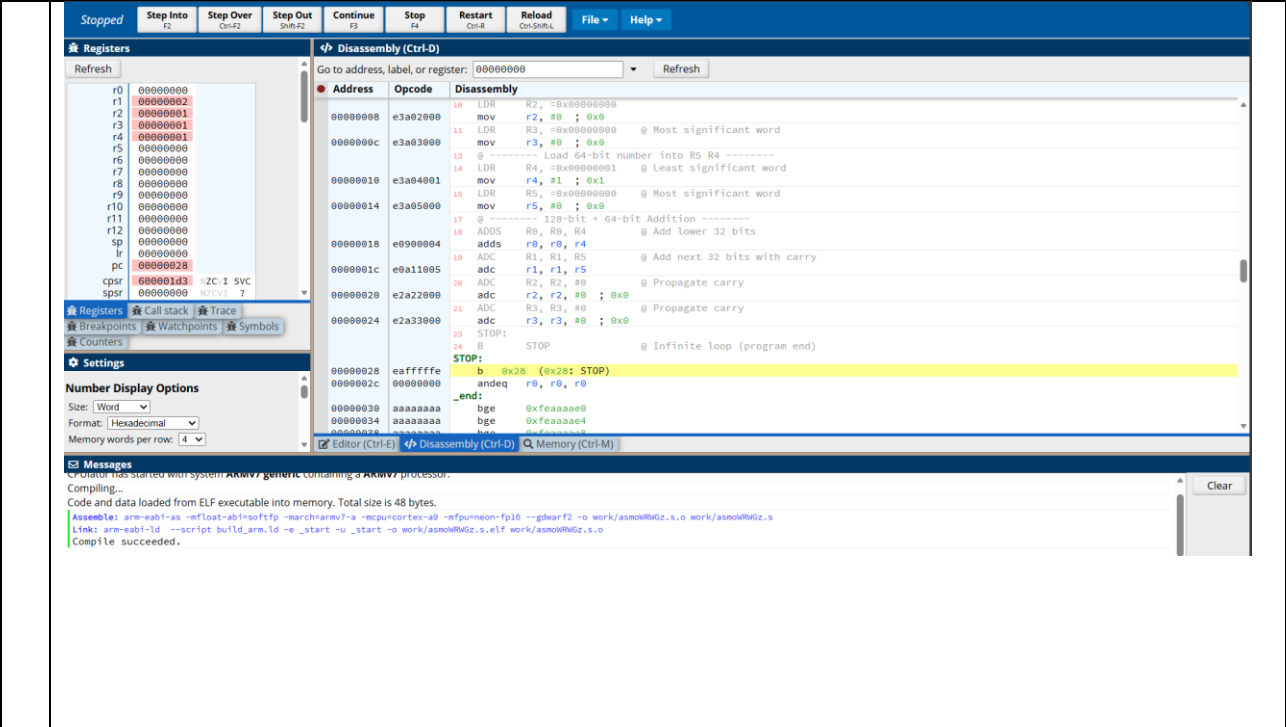
Department of Computer Science & Engineering

Microprocessor & Computer Architecture

UE24CS251B

MPCA Lab Week3- Submission

Name: DHRITI KAMANI		SRN: PES1UG24AM085	Section: B
1	Write an ALP to perform Addition for of two numbers of sizes 64 bit and 128 bit and save the result in register (reuse the register to store the result) Hint: R3 R2 R1 R0 + R5 R4 Program and Results Screenshot: <pre>.global _start _start: LDR R0, =0xFFFFFFFF LDR R1, =0x00000001 LDR R2, =0x00000000 LDR R3, =0x00000000 LDR R4, =0x00000001 LDR R5, =0x00000000 ADDS R0, R0, R4 ADC R1, R1, R5 ADC R2, R2, #0 ADC R3, R3, #0 STOP: B STOP</pre>		



2 Write a program to find the factorial of a given number.

Program and Results Screenshot:

```

.global _start

_start:
    MOV    R1, #5
    MOV    R0, #1

FACT_LOOP:
    CMP    R1, #1
    BLE    STOP

    MUL    R0, R0, R1
    SUB    R1, R1, #1
    B      FACT_LOOP

STOP:
    B      STOP
  
```

Registers

Refresh

r0	00000078
r1	00000001
r2	00000000
r3	00000000
r4	00000000
r5	00000000
r6	00000000
r7	00000000
r8	00000000
r9	00000000
r10	00000000
r11	00000000
r12	00000000
sp	00000000
lr	00000000
pc	0000001c
cpsr	600001d3 NZCVI SVC
spsr	00000000 NZCVI ?

Registers

Call stack

Trace

Breakpoints

Watchpoints

Symbols

Counters

Settings

Number Display Options

Size: Word

Format: Hexadecimal

Memory words per row: 4

Disassembly (Ctrl-D)

Go to address, label, or register: 00000000 Refresh

Address	Opcode	Disassembly
ffffff4	aaaaaaaa	bge 0xfeaaaaa4
ffffff8	aaaaaaaa	bge 0xfeaaaaa8
ffffffc	aaaaaaaa	bge 0xfeaaaaac
		2 _start:
		4 .global _start
		6 _start:
		8 _start:
00000000	e3a01005	7 mov r1, #5 ; 0x5
00000004	e3a00001	8 mov r0, #1 ; 0x1
		10 FACT_LOOP:
		11 cmp r1, #1 ; 0x1
		12 BLE STOP
0000000c	da000002	13 btle 0x1c (0x1c: STOP)
00000010	e0000190	14 mul r0, r0, r1
00000014	e2411001	15 sub r1, r1, #1 ; 0x1
		16 FACT_LOOP
00000018	ea000000	17 b 0x8 (0x8: FACT_LOOP)
		18 STOP:
		19 b STOP
0000001c	ea000000	20 STOP:
		21 b 0x1c (0x1c: STOP)
		22 _end:
00000020	aaaaaaaa	23 bge 0xfeaaaaad0
00000024	aaaaaaaa	24 bge 0xfeaaaaad4
00000028	aaaaaaaa	25 bge 0xfeaaaaad8
0000002c	aaaaaaaa	26 bge 0xfeaaaaadc

Editor (Ctrl-E)

Disassembly (Ctrl-D)

Memory (Ctrl-M)

Messages

CPULab has started with system ARMv7 generic containing a ARMv7 processor.

Compiling...

Code and data loaded from ELF executable into memory. Total size is 32 bytes.

```

Assemble: arm-eabi-as -mfloat-abi=softfp -march=armv7-a -mcpu=cortex-a9 -mcpu=neon-fp16 --gdwarf2 -o work/asmJ0TMgm.s.o work/asmJ0TMgm.s
Link: arm-eabi-ld --script build_arm.ld -e _start -u _start -o work/asmJ0TMgm.s.elf work/asmJ0TMgm.s.o
Compile succeeded.

```

3 Write an ALP to find the number of zeroes, positive and negative numbers in a given array

Program and Results Screenshot:

```
.global _start
```

```
_start:
```

```

LDR R0, =ARRAY
MOV R1, #5
MOV R2, #0
MOV R3, #0
MOV R4, #0

```

```
LOOP:
```

```

CMP R1, #0
BEQ STOP

```

```

LDR R5, [R0], #4
CMP R5, #0
BEQ ZERO
BGT POS
BLT NEG

```

```
ZERO:
```

```
ADD R2, R2, #1
```

B NEXT

POS:

ADD R3, R3, #1

B NEXT

NEG:

ADD R4, R4, #1

NEXT:

SUB R1, R1, #1

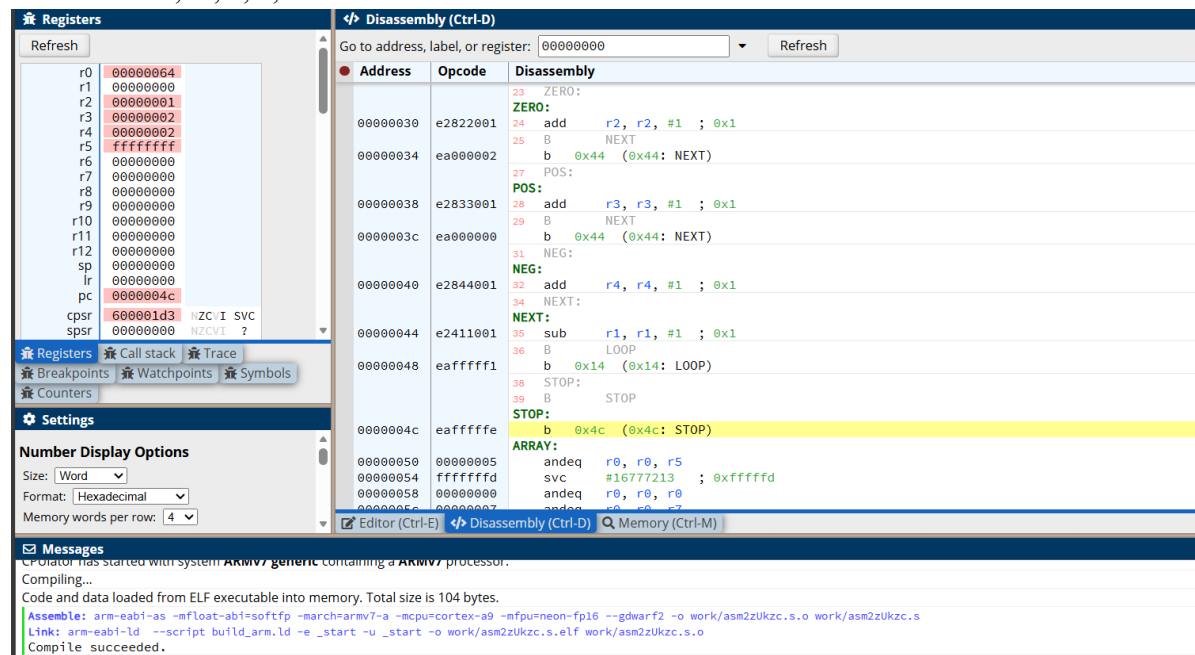
B LOOP

STOP:

B STOP

ARRAY:

.word 5, -3, 0, 7, -1



The screenshot shows a debugger interface with three main panes:

- Registers:** A list of registers (r0-r15, sp, lr, pc, cpsr, spsr) with their current values. The PC register is highlighted at 0000004c.
- Disassembly (Ctrl-D):** A table showing the assembly code being executed. The code includes labels like ZERO, POS, NEG, NEXT, LOOP, and STOP, with corresponding opcodes and addresses. The instruction at address 0000004c is highlighted in yellow.
- Messages:** A log of system messages, including compilation and linking details for the ARMv7 generic processor.

4 Write a program to perform 3X3 matrix addition.

Program and Results Screenshot:

.global _start

_start:

LDR R0, =MAT1

LDR R1, =MAT2

```
LDR  R2, =RES
MOV  R3, #9
```

LOOP:

```
CMP  R3, #0
BEQ  STOP
```

```
LDR  R4, [R0], #4
LDR  R5, [R1], #4
ADD  R6, R4, R5
STR  R6, [R2], #4
```

```
SUB  R3, R3, #1
B    LOOP
```

STOP:

```
B    STOP
```

MAT1:

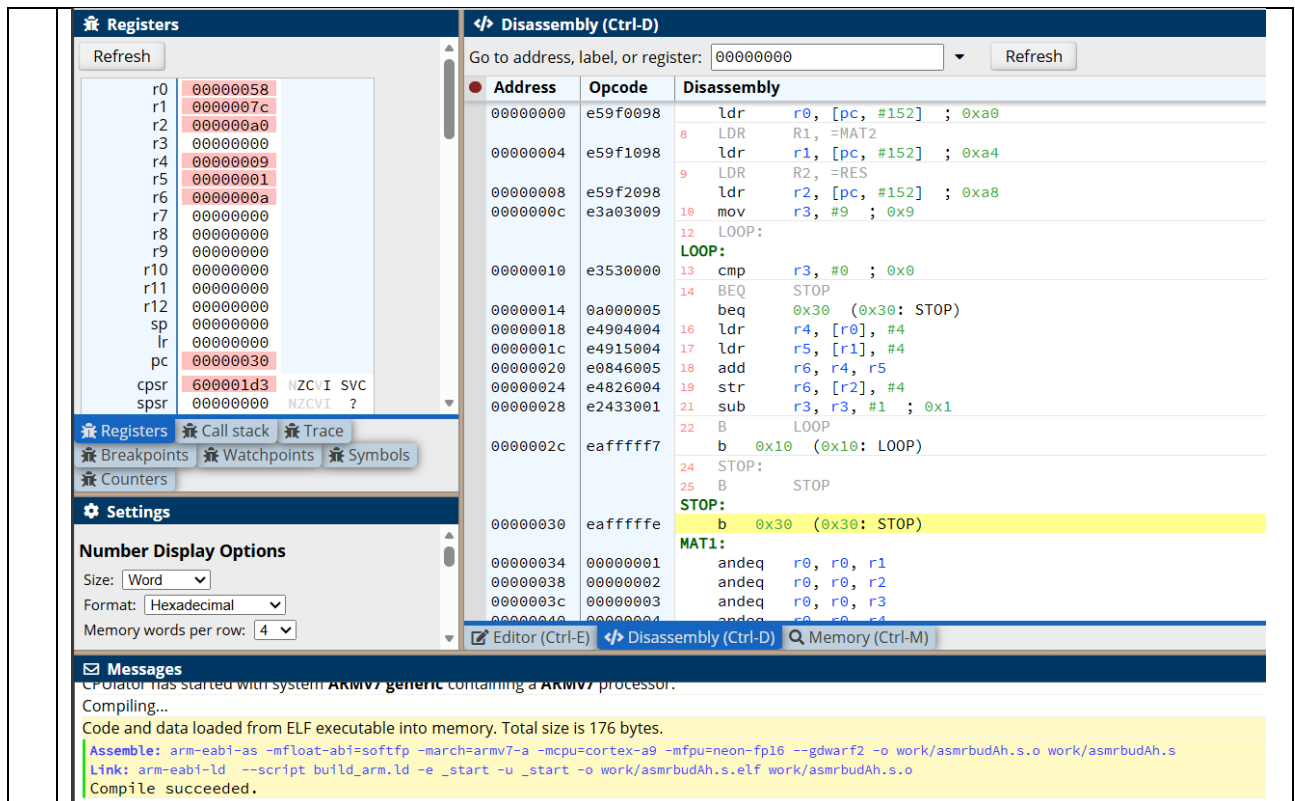
```
.word 1, 2, 3
.word 4, 5, 6
.word 7, 8, 9
```

MAT2:

```
.word 9, 8, 7
.word 6, 5, 4
.word 3, 2, 1
```

RES:

```
.space 36
```



The screenshot shows a debugger window with the following sections:

- Registers:** A list of registers (r0 to r15, sp, lr, pc, cpsr, spsr) with their current values. The pc register is highlighted with the value 00000030.
- Disassembly (Ctrl-D):** A table showing the disassembly of instructions at various memory addresses. The instructions include LDR, MOV, CMP, BEQ, and STOP. The address 00000030 is highlighted.
- Messages:** A log of messages from the processor, including "CP: Processor has started with system AKM7 generic containing a AKM7 processor.", "Compiling...", "Code and data loaded from ELF executable into memory. Total size is 176 bytes.", and "Compile succeeded."

5 Assignment

Write a program in ARM7TDMI-ISA to find the sum of N data items at alternate [odd or even positions] locations in the memory. Store the result in the memory location.

- Use Pre-indexing addressing mode
- Use Post- Indexing addressing mode
- Use Auto-indexing addressing mode

Program and Results Screenshot:

```

a. .global _start

_start:
    LDR    R0, =ARR
    MOV    R1, #3
    MOV    R2, #0

LOOP1:
    LDR    R3, [R0, #8]!
    ADD    R2, R2, R3
    SUB    R1, R1, #1
    BNE    LOOP1

    LDR    R4, =RESULT
    STR    R2, [R4]
  
```

STOP:

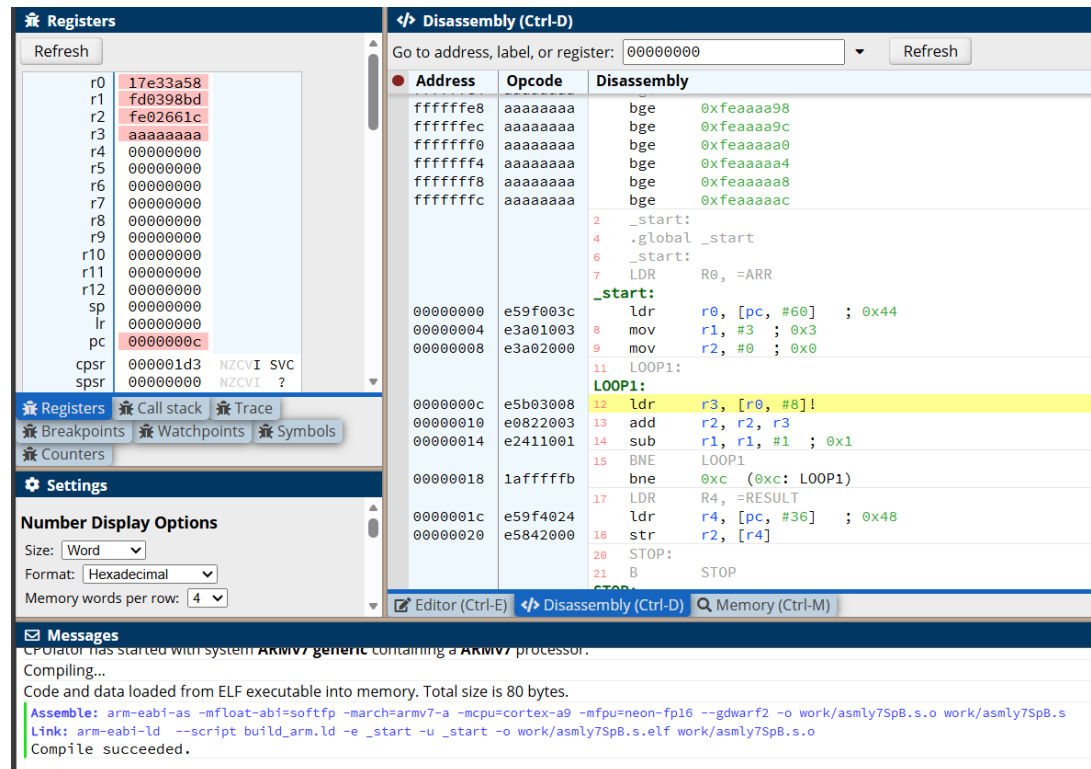
B STOP

ARR:

.word 1, 2, 3, 4, 5, 6

RESULT:

.word 0



The screenshot shows a debugger interface with the following components:

- Registers:** A list of registers (r0-r15, sp, lr, pc, cpsr, spsr) with their current values. r0 is highlighted with a red background.
- Disassembly (Ctrl-D):** A table showing the disassembly of the code. The address column shows the memory address, the opcode column shows the instruction, and the disassembly column shows the instruction in assembly syntax. The instructions are:
 - ffffffe8: bge 0xfeaaaa98
 - ffffffec: bge 0xfeaaaa9c
 - fffffff0: bge 0xfeaaaaa0
 - fffffff4: bge 0xfeaaaaa4
 - fffffff8: bge 0xfeaaaaa8
 - fffffffc: bge 0xfeaaaaac
 - 2: _start:
 - 4: .global _start
 - 6: _start:
 - 7: LDR R0, =ARR
 - 8: _start:
 - 9: ldr r0, [pc, #60] ; 0x44
 - 10: mov r1, #3 ; 0x3
 - 11: mov r2, #0 ; 0x0
 - 12: LOOP1:
 - 13: ldr r3, [r0, #8]!
 - 14: add r2, r2, r3
 - 15: sub r1, r1, #1 ; 0x1
 - 16: BNE LOOP1
 - 17: bne 0xc (0xc: LOOP1)
 - 18: LDR R4, =RESULT
 - 19: ldr r4, [pc, #36] ; 0x48
 - 20: str r2, [r4]
 - 21: STOP:
 - 22: B STOP
- Messages:** A list of messages showing the compilation process. The messages are:
 - Compiler has started with system ARMv7-generic containing a ARMv7 processor.
 - Compiling...
 - Code and data loaded from ELF executable into memory. Total size is 80 bytes.
 - Assemble: arm-eabi-as -mfloat-abi=softfp -march=armv7-a -mcpu=cortex-a9 -mcpu=neon-fp16 --gdwarf2 -o work/asmly7SpB.s.o work/asmly7SpB.s
 - Link: arm-eabi-ld --script build_arm.ld -e _start -u _start -o work/asmly7SpB.s.elf work/asmly7SpB.s.o
 - Compile succeeded.

b. .global _start

_start:

LDR R0, =ARR

MOV R1, #3

MOV R2, #0

LOOP2:

LDR R3, [R0], #8

ADD R2, R2, R3

SUB R1, R1, #1

BNE LOOP2

LDR R4, =RESULT

STR R2, [R4]

STOP:

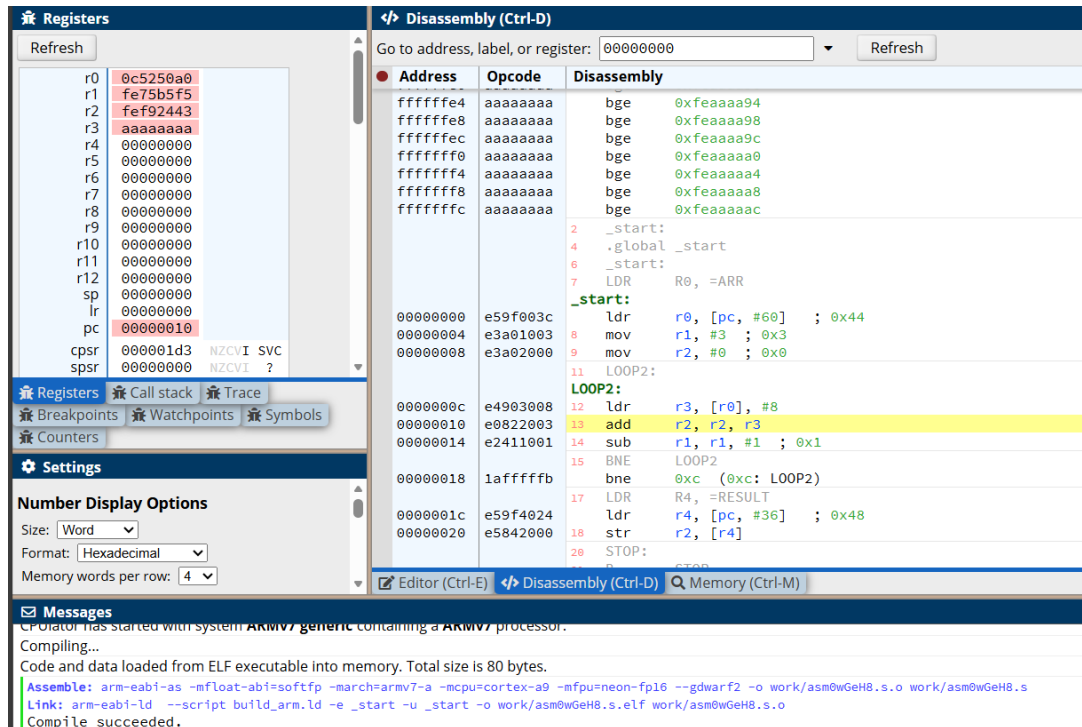
B STOP

ARR:

.word 1, 2, 3, 4, 5, 6

RESULT:

.word 0



Registers

Register	Value
r0	0c5250a0
r1	fe75b5f5
r2	fef92443
r3	aaaaaaaa
r4	00000000
r5	00000000
r6	00000000
r7	00000000
r8	00000000
r9	00000000
r10	00000000
r11	00000000
r12	00000000
sp	00000000
lr	00000000
pc	00000010
cpsr	000001d3 NZCVI SVC
spsr	00000000 NZCVI ?

Disassembly (Ctrl-D)

Go to address, label, or register: 00000000 Refresh

Address	Opcode	Disassembly
ffffffe4	aaaaaaaa	bge 0xfeaaaa94
ffffffe8	aaaaaaaa	bge 0xfeaaaa98
ffffffec	aaaaaaaa	bge 0xfeaaaa9c
fffffff0	aaaaaaaa	bge 0xfeaaaaa0
fffffff4	aaaaaaaa	bge 0xfeaaaaa4
fffffff8	aaaaaaaa	bge 0xfeaaaaa8
fffffffc	aaaaaaaa	bge 0xfeaaaaac
2		_start:
4		.global _start
6		_start:
7		LDR R0, =ARR
8		_start:
9		ldr r0, [pc, #60] ; 0x44
10		mov r1, #3 ; 0x3
11		mov r2, #0 ; 0x0
12		LOOP2:
13		ldr r3, [r0], #8
14		add r2, r2, r3
15		sub r1, r1, #1 ; 0x1
16		BNE LOOP2
17		bne 0xc (0xc: LOOP2)
18		LDR R4, =RESULT
19		ldr r4, [pc, #36] ; 0x48
20		str r2, [r4]
21		STOP:

Messages

CPUIlator has started with system **ARMv7-Generic** containing a **ARMv7** processor.

Compiling...

Code and data loaded from ELF executable into memory. Total size is 80 bytes.

Assemble: arm-eabi-as -mfloat-abi=softfp -march=armv7-a -mcpu=cortex-a9 -mfp=neon-fp16 --gdwarf2 -o work/asm0wGeH8.s.o work/asm0wGeH8.s

Link: arm-eabi-ld --script build_arm.ld -e _start -u _start -o work/asm0wGeH8.s.elf work/asm0wGeH8.s.o

Compile succeeded.

c. .global _start

_start:

LDR R0, =ARR

MOV R1, #3

MOV R2, #0

MOV R6, #8

LOOP3:

LDR R3, [R0, R6]

ADD R2, R2, R3

ADD R0, R0, R6

SUB R1, R1, #1

BNE LOOP3

LDR R4, =RESULT

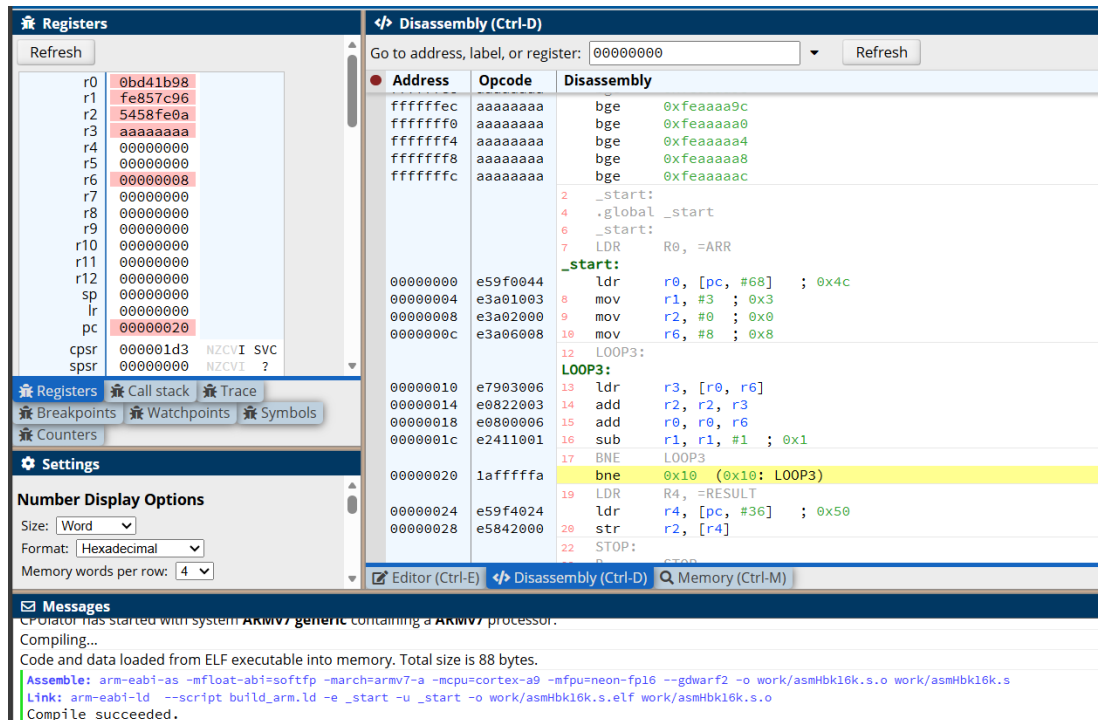
STR R2, [R4]

```

STOP:
    B    STOP

ARR:
    .word 1, 2, 3, 4, 5, 6

RESULT:
    .word 0
  
```



Registers

Register	Value
r0	0bd41b98
r1	fe857c96
r2	5458fe0a
r3	aaaaaaaa
r4	00000000
r5	00000000
r6	00000000
r7	00000000
r8	00000000
r9	00000000
r10	00000000
r11	00000000
r12	00000000
sp	00000000
lr	00000000
pc	00000020
cpsr	000001d3 NZCVI SVC
spsr	00000000 NZCVI ?

Disassembly (Ctrl-D)

Address	Opcode	Disassembly
ffffffec	aaaaaaaa	bge 0xfeaaaa9c
fffffffd	aaaaaaaa	bge 0xfeaaaaa0
fffffffe	aaaaaaaa	bge 0xfeaaaaa4
fffffff0	aaaaaaaa	bge 0xfeaaaaa8
fffffff4	aaaaaaaa	bge 0xfeaaaaac
00000000	e59f0044	ldr r0, [pc, #68] ; 0x4c
00000004	e3a01003	mov r1, #3 ; 0x3
00000008	e3a02000	mov r2, #0 ; 0x0
0000000c	e3a06008	mov r6, #8 ; 0x8
00000010	e7903006	ldr r3, [r0, r6]
00000014	e0822003	add r2, r2, r3
00000018	e0800006	add r0, r0, r6
0000001c	e2411001	sub r1, r1, #1 ; 0x1
00000020	1affffffa	bne 0x10 (0x10: LOOP3)
00000024	e59f4024	ldr r4, [pc, #36] ; 0x50
00000028	e5842000	str r2, [r4]
0000002c	00000000	STOP

Messages

```

C-Compiler has started with system ARMv7-generic containing a ARMv7 processor.
Compiling...
Code and data loaded from ELF executable into memory. Total size is 88 bytes.
Assemble: arm-eabi-as -mfloat-abi=softfp -march=armv7-a -mcpu=cortex-a9 -mfpu=neon-fp16 --gdwarf2 -o work/asmHbk16k.s.o work/asmHbk16k.s
Link: arm-eabi-ld --script build_arm.ld -e _start -u _start -o work/asmHbk16k.s.elf work/asmHbk16k.s.o
Compile succeeded.
  
```

6 Assignment:

Write an ALP to perform 2's complement, Only using MOV and RSB instruction.

Program and Results Screenshot:

```

.global _start

_start:
    MOV    R1, #25
    RSB    R0, R1, #0

STOP:
    B      STOP
  
```

Registers

Refresh

r0	ffffffe7
r1	00000019
r2	00000000
r3	00000000
r4	00000000
r5	00000000
r6	00000000
r7	00000000
r8	00000000
r9	00000000
r10	00000000
r11	00000000
r12	00000000
sp	00000000
lr	00000000
pc	00000008
cpsr	000001d3 NZCVI SVC
spsr	00000000 NZCVI ?

Registers

Call stack

Trace

Breakpoints

Watchpoints

Symbols

Counters

Settings

Number Display Options

Size: Word

Format: Hexadecimal

Memory words per row: 4

Disassembly (Ctrl-D)

Go to address, label, or register: 00000000 Refresh

Address	Opcode	Disassembly
ffffffe8	aaaaaaaa	bge 0xfeaaaa98
ffffffec	aaaaaaaa	bge 0xfeaaaa9c
fffffff0	aaaaaaaa	bge 0xfeaaaaa0
fffffff4	aaaaaaaa	bge 0xfeaaaaa4
fffffff8	aaaaaaaa	bge 0xfeaaaaa8
fffffffc	aaaaaaaa	bge 0xfeaaaaac
2 _start:		
4 .global _start		
6 _start:		
7 mov r1, #25 ; 0x19		
00000000	e3a01019	8 rsb r0, r1, #0 ; 0x0
00000004	e2610000	10 STOP:
11 B STOP		
STOP:		
00000008	eafffffe	b 0x8 (0x8: STOP)
0000000c	00000000	andeq r0, r0, r0
_end:		
00000010	aaaaaaaa	bge 0xfeaaaaac0
00000014	aaaaaaaa	bge 0xfeaaaaac4
00000018	aaaaaaaa	bge 0xfeaaaaac8
0000001c	aaaaaaaa	bge 0xfeaaaaacc
00000020	aaaaaaaa	bge 0xfeaaaaad0
00000024	aaaaaaaa	bge 0xfeaaaaad4
00000028	aaaaaaaa	bge 0xfeaaaaad8
0000002c	aaaaaaaa	bge 0xfeaaaaadc
00000030	aaaaaaaa	bge 0xfeaaaaae0

Editor (Ctrl-E)

Disassembly (Ctrl-D)

Memory (Ctrl-M)

Messages

CP0rator has started with system **AKMV7 generic** containing a **AKMV7** processor.

Compiling...

Code and data loaded from ELF executable into memory. Total size is 16 bytes.

Assemble: arm-eabi-as -mfloat-abi=softfp -march=armv7-a -mcpu=cortex-a9 -mfp=neon-fp16 --gdwarf2 -o work/asmYnK1pj.s.o work/asmYnK1pj.s

Link: arm-eabi-ld --script build_arm.ld -e _start -u _start -o work/asmYnK1pj.s.elf work/asmYnK1pj.s.o

Compile succeeded.